```
      </ul>
    </div>
  <% end %>

  <p>
    <%= f.label :title %><br>
    <%= f.text_field :title %>
  </p>

  <p>
    <%= f.label :text %><br>
    <%= f.text_area :text %>
  </p>

  <p>
    <%= f.submit %>
  </p>

<% end %>

<%= link_to 'Back', articles_path %>
```

This time we point the form to the `update` action, which is not defined yet but will be very soon.
The `method: :patch` option tells Rails that we want this form to be submitted via the `PATCH` HTTP method which is the HTTP method you're expected to use to **update** resources according to the REST protocol.
The first parameter of `form_for` can be an object, say, `@article` which would cause the helper to fill in the form with the fields of the object. Passing in a symbol (`:article`) with the same name as the instance variable (`@article`) also automagically leads to the same behavior. This is what is happening here. More details can be found in form_for documentation.
Next, we need to create the `update` action in `app/controllers/articles_controller.rb`. Add it between the `create` action and the `private` method:

```
def create
  @article = Article.new(article_params)

  if @article.save
    redirect_to @article
  else
    render 'new'
  end
end

def update
  @article = Article.find(params[:id])

  if @article.update(article_params)
    redirect_to @article
  else
    render 'edit'
  end
end

private
  def article_params
    params.require(:article).permit(:title, :text)
  end
```

The new method, `update`, is used when you want to update a record that already exists, and it accepts a hash containing the attributes that you want to update. As before, if there was an error updating the article we want to show the form back to the user.
We reuse the `article_params` method that we defined earlier for the create action.